

Mini SaaS Template Store

Forma - a full-stack template discovery and favorites app

Candidate	David Shalom M
Email	davidshalomswe@gmail.com
Phone	7539943015
Assessment	Full Stack Web Developer Intern - Mini SaaS Template Store
Submitted	9 October 2026

A clean, responsive template catalog where users can create an account, browse and filter templates, and save a personal favorites list.

Live website forma-template-store-client.vercel.app

Source repository github.com/davidShalom-git/fullstack-intern-task

01 / Solution overview

What the application does

Forma presents a small catalog of website templates and gives each registered user a private shortlist of favorites. Visitors can explore the catalog without signing in. Account features are available through dedicated registration and login pages.

Assessment requirements covered

Requirement	Implementation
Account registration and login	Express endpoints validate name, email, and password; passwords are hashed with bcryptjs; successful login returns a signed JWT.
Template collection and API	Mongoose Template model with name, description, thumbnail URL, and category. Public list and detail endpoints are available.
Favorites	Signed-in users can add, list, and remove favorites. Each favorite links a user to a template.
React pages and API integration	React Router pages for /register, /login, /templates, and /favorites. Axios calls the API; protected routes limit access to favorites.
Seed data	Seven starter templates are kept available. Seeding adds missing examples without replacing records that are already in the database.
Validation and errors	Request validation, appropriate HTTP status codes, and JSON error messages are handled in the API.

Additional features

The catalog includes search and category filtering. Users can log out, see which templates are already favorited, and use a protected favorites page. The SaaS category includes a purpose-made dashboard preview with project activity, usage cards, and a team workspace. The app is deployed with a Vercel frontend and API, connected to MongoDB Atlas.

How the pieces work together

Layer	Responsibility
React + Zustand	Renders the pages and forms. Zustand keeps the current session available to the UI; the JWT and basic user profile are stored in local storage as required by the assessment.
Axios API client	Uses the configured API base URL and adds the Bearer token to requests when the user is signed in.
Express API	Validates inputs, handles authentication, returns JSON, and mounts routes for users, templates, and favorites.
JWT middleware	Verifies the token before protected endpoints read or change the signed-in user's favorites.
Mongoose + MongoDB	Stores user, template, and favorite documents. The live Vercel API connects to MongoDB Atlas using server-side environment variables.

Data model

Collection	Key information
users	Name, normalized email, and password hash. Passwords are never stored as plain text.
templates	Name, description, thumbnail URL, and category.
favorites	References to a user and a template, plus timestamps. A unique compound index prevents duplicate user/template pairs.

Key design choices

The catalog is small, so text search and category filtering run in the browser after the template list is loaded. API responses expose MongoDB identifiers as plain id strings so the UI does not depend on MongoDB-specific fields. Authentication logic and route groups are kept in separate files to make the flow easier to follow.

Routes and deployment

Method	Route	Access and purpose
POST	/api/auth/register	Public - create account and issue token
POST	/api/auth/login	Public - verify credentials and issue token
GET	/api/auth/me	Bearer token - return current profile
GET	/api/templates	Public - list templates
GET	/api/templates/:id	Public - template details
GET	/api/favorites	Bearer token - list current user favorites
POST / DELETE	/api/favorites/:templateId	Bearer token - add or remove a favorite
GET	/api/health	Public - API health status

Deployment

Part	Location	Configuration
Frontend	Vercel	React app; API base URL supplied through VITE_API_URL.
API	Vercel	Express app; MongoDB URI, JWT secret, and allowed frontend origin are server environment settings.
Database	MongoDB Atlas	Hosted database; no local database is required for the deployed application.

Verification completed

The client production build completed successfully. The deployed frontend returned HTTP 200 for the main application routes. The live API health endpoint returned HTTP 200, and GET /api/templates returned six templates. The full authenticated registration and favorites journey should also be demonstrated during review.

Local setup

The repository README describes the local setup. In brief, the client and server are separate Node.js applications; the server needs MONGODB_URI and a private JWT_SECRET in its environment, and the client uses VITE_API_URL to select the API. Environment files and real credentials are excluded from version control.

Repository README: [setup](#), [project structure](#), and [further design notes](#).

Thank you for reviewing my assessment. I would be glad to walk through the implementation and explain my decisions.